

[28] Chroma: The open-source AI application database 총정리

조직: Chroma (오픈소스 프로젝트)

형태: 오픈소스 임베딩/AI 애플리케이션 데이터베이스

웹사이트: <https://www.trychroma.com/>

공개/업데이트: 2024

요약

Chroma 는 AI 애플리케이션 개발자를 위해 설계된 경량의 오픈소스 벡터 데이터베이스이다. 특히 개발 단계에서 프로토타이핑과 소규모 배포에 최적화되어 있으며, 간단한 Python API 를 통해 빠르게 벡터 검색 기능을 통합할 수 있다. Chroma 의 강점은 사용의 단순성과 유연성에 있다. 로컬 파일 시스템, 인메모리 저장소, 또는 분산 모드로의 배포가 모두 가능하여 개발부터 프로덕션까지의 여정을 지원한다. LLM 기반 애플리케이션, 특히 RAG 시스템에서 외부 지식 베이스 역할을 담당한다. 메타데이터 필터링, 다양한 거리 메트릭, 그리고 쉬운 확장성을 제공하여 초기 AI 서비스 개발의 진입 장벽을 낮춘다.

상세분석

28.1 주요 문제점과 설계 목표

기존 벡터 데이터베이스 솔루션의 한계와 Chroma 의 해결 방안:

- **복잡성:** Milvus, Weaviate 등 기존 시스템의 설정과 배포가 복잡
 - Chroma 해결책: 최소 구성으로 5 분 내 시작 가능
- **진입 장벽:** 소규모 팀이나 초기 단계 스타트업의 높은 인프라 비용
 - Chroma 해결책: 로컬 개발 모드 완전 무료, 메모리 기반 배포
- **프로토타이핑 속도:** 빠른 개념 검증의 필요성
 - Chroma 해결책: 즉시 사용 가능한 Python API
- **확장성 필요:** 개발 중 쉽게 확장할 수 있는 아키텍처
 - Chroma 해결책: 로컬 → 분산 모드로의 무경계 전환

28.2 핵심 기능 및 아키텍처

배포 모드

1. 인메모리 모드 (In-Memory):

- 프로그램 시작 시 데이터 초기화
- 매우 빠른 성능
- 재시작 시 데이터 손실
- 개발 및 테스트에 적합

2. 영구 로컬 저장소 모드 (Persistent Local):

- SQLite + DuckDB 로 메타데이터와 벡터 저장
- 로컬 파일 시스템 기반 벡터 저장
- 소규모 애플리케이션(수백만 벡터)에 적합
- 재시작 후 데이터 유지

3. 클라이언트-서버 모드 (Client-Server):

- Chroma 서버를 별도 프로세스로 실행
- REST API 를 통한 통신
- 여러 클라이언트의 동시 접근 지원
- 초기 분산 배포 단계

4. 분산 모드 (Distributed):

- 쿠버네티스 기반 배포
- 수평 확장
- 고가용성 구성

핵심 API 설계

간결하고 직관적인 Python 인터페이스:

```
# 컬렉션 생성
collection = client.create_collection(name="documents")

# 벡터 추가
collection.add(
    ids=["id1", "id2"],
    embeddings=[[1.1, 2.3], [4.5, 6.9]],
    metadatas=[{"source": "doc1"}, {"source": "doc2"}],
    documents=["text1", "text2"]
)

# 검색
results = collection.query(
    query_embeddings=[[1.1, 2.3]],
    n_results=10,
    where={"source": "doc1"}
)
```

주요 메서드:

- `create_collection()`: 새 컬렉션 생성
- `add()`: 벡터와 메타데이터 추가
- `delete()`: 벡터 삭제
- `update()`: 벡터 업데이트
- `query()`: 유사 벡터 검색
- `get()`: 특정 벡터 조회

메타데이터 필터링

메타데이터와 벡터 유사도의 결합 검색:

```
results = collection.query(
    query_embeddings=[[1.1, 2.3]],
    n_results=10,
    where={
        "$and": [
            {"category": "technology"},
            {"year": {"$gte": 2020}}
        ]
    }
)
```

필터 연산자: \$and, \$or, \$eq, \$ne, \$gt, \$gte, \$lt, \$lte, \$in, \$nin

임베딩 모델 통합

내장 임베딩:

- Chroma 의 기본 임베딩 함수로 자동 처리
- 문서 자체를 저장하고 쿼리로 자동 임베딩

외부 모델 통합:

- OpenAI, HuggingFace, Ollama 등과의 통합
- 커스텀 임베딩 함수 작성 가능

28.3 저장소 구현

메타데이터 저장소

SQLite 또는 DuckDB:

- 메타데이터 필터링을 위한 SQL 쿼리 지원
- ACID 트랜잭션 보장
- 작은 데이터셋에 최적화

벡터 저장소

로컬 파일 시스템 (persistent 모드):

- 벡터를 효율적인 바이너리 포맷으로 저장
- NumPy 또는 Parquet 형식 활용

메모리 (in-memory 모드):

- 빠른 접근

외부 저장소 (분산 모드):

- 오브젝트 스토리지 (S3, GCS) 연동 가능

28.4 검색 성능 최적화

인덱싱 전략

Chroma 는 단순성을 위해 제한된 인덱싱 옵션:

- **정확한 검색 (Exact Search):** 모든 벡터와 비교 (작은 컬렉션용)
- **해시 기반 근사 (Locality Sensitive Hashing):** 대규모 벡터 대응

캐싱

- 최근 쿼리 결과 캐싱
- 메타데이터 쿼리 캐싱

28.5 본 논문과의 관계

Exqutor 은 텍스트 쿼리를 벡터 검색으로 변환하는 하이브리드 시스템이다. Chroma 는 Exqutor 의 벡터 저장 및 검색 백엔드로서 두 가지 역할을 한다:

1. **개발 및 프로토타이핑:** Exqutor 의 초기 구현과 테스트에 이상적인 가벼운 벡터 DB
2. **소규모 배포:** 엔터프라이즈 요구사항이 낮은 환경에서 경량의 벡터 검색 인프라 제공

또한 Chroma 의 간결한 API 설계는 Exqutor 이 제공해야 할 사용자 인터페이스의 간소화와 직관성에 영감을 줄 수 있다. 특히 메타데이터 필터링과 벡터 검색의 통합 방식은 Exqutor 의 하이브리드 검색 전략과 직접 연계된다.

추가 제기 문제

1. **확장성 한계:** SQLite/DuckDB 기반의 메타데이터 저장소가 어느 정도의 벡터 개수와 쿼리 빈도까지 효율적으로 처리할 수 있는가?
2. **정확한 vs 근사 검색의 자동 선택:** 컬렉션 크기에 따라 자동으로 인덱싱 전략을 전환하는 기준은? 전환 과정의 오버헤드는?
3. **멀티 클라이언트 동시성:** 클라이언트-서버 모드에서 다양한 클라이언트의 동시 삽입/삭제/검색 시 일관성 보장 수준은?
4. **메모리 효율성:** 인메모리 모드에서 대규모 벡터 데이터셋(수억 개)을 저장할 때 메모리 압축 기법은?
5. **분산 모드 성능:** 분산 모드에서 네트워크 지연이 단일 노드 모드 대비 검색 성능에 얼마나 영향을 미치는가?
6. **임베딩 모델 버전 관리:** 임베딩 모델이 변경될 때 기존 벡터의 재생성과 재인덱싱 전략은?